

# OpenCode 项目完整架构分析文档

---

OpenCode 是一个基于 TypeScript 的 AI 编程助手，采用事件驱动架构和插件化设计。

文档版本: 1.0 分析日期: 2026-02-09 项目路径: `/mnt/c/dev/tmp/opencode`

---

## 目录

1. 项目概览
  2. 技术栈
  3. 目录结构
  4. 架构设计原则
  5. 核心模块详解
  6. 请求处理完整流程
  7. 数据流与模块关系
  8. 设计模式总结
- 

## 1. 项目概览

OpenCode 是一个功能强大的 AI 编程助手，类似于 Claude Code，提供了完整的代码编辑、命令执行、文件搜索等功能。其核心特点包括：

- **多 LLM 提供商支持**: 支持 OpenAI、Anthropic、Google、本地模型等
  - **插件化架构**: 支持自定义工具、Agent 和提供商
  - **流式处理**: 实时显示 AI 响应和工具执行状态
  - **细粒度权限控制**: 基于规则的权限管理系统
  - **MCP 协议支持**: 集成 Model Context Protocol
-

## 2. 技术栈

| 技术            | 说明            |
|---------------|---------------|
| 运行时           | Bun           |
| 语言            | TypeScript    |
| <b>Web</b> 框架 | Hono          |
| <b>AI SDK</b> | Vercel AI SDK |
| <b>CLI</b> 框架 | Yargs         |
| 数据验证          | Zod           |
| 存储            | 文件系统 + JSON   |
| 搜索            | Ripgrep       |
| 消息队列          | 自定义事件系统 (Bus) |

### 3. 目录结构

```

packages/opencode/
├─ src/
│   ├─ index.ts           # 主入口点
│   │
│   │
│   ├─ cli/               # CLI 命令行接口
│   │   ├─ cmd/           # 命令定义
│   │   │   ├─ run.ts     # 运行命令
│   │   │   ├─ auth.ts    # 认证管理
│   │   │   ├─ agent.ts   # Agent 管理
│   │   │   ├─ generate.ts # 生成命令
│   │   │   ├─ serve.ts   # 服务命令
│   │   │   └─ session.ts # 会话管理
│   │   └─ ...
│   │   ├─ error.ts       # 错误格式化
│   │   └─ ui.ts          # UI 组件
│   │
│   ├─ agent/             # AI 智能体管理
│   │   ├─ agent.ts       # Agent 定义和配置
│   │   └─ generate.ts    # Agent 生成
│   │
│   ├─ session/           # 会话管理
│   │   ├─ session.ts     # 会话创建和管理
│   │   ├─ message-v2.ts  # 消息数据结构
│   │   ├─ llm.ts         # LLM 通信
│   │   ├─ processor.ts   # 会话处理器 (核心)
│   │   ├─ system.ts      # 系统提示词构建
│   │   ├─ prompt.ts      # 提示词处理入口
│   │   └─ prompt/        # 提示词模板
│   │       ├─ codex_header.txt # 核心指令
│   │       ├─ anthropic.txt   # Anthropic 特定提示
│   │       ├─ beast.txt       # GPT 特定提示
│   │       └─ gemini.txt      # Gemini 特定提示
│   │       └─ ...
│   │   └─ retry.ts         # 重试机制
│   │       └─ status.ts    # 会话状态
│   │
│   └─ tool/               # 工具系统

```

```

| | └─ tool.ts          # 工具基础接口
| | └─ registry.ts      # 工具注册中心
| | └─ truncation.ts    # 输出截断
| | └─ builtin/         # 内置工具
| |   └─ read.ts        # 文件读取
| |   └─ write.ts       # 文件写入
| |   └─ edit.ts        # 文件编辑
| |   └─ bash.ts        # 命令执行
| |   └─ glob.ts        # 文件模式匹配
| |   └─ grep.ts        # 内容搜索
| |   └─ task.ts        # 任务管理
| |   └─ websearch.ts   # 网络搜索
| |   └─ webfetch.ts    # 网页抓取
| |   └─ skill.ts       # 技能执行
| |   └─ ...
| | └─ context.ts       # 工具上下文
| |
| └─ provider/          # LLM 提供商管理
|   └─ provider.ts      # 提供商接口定义
|   └─ transform.ts     # 消息转换
|   └─ builtin/         # 内置提供商
|
| └─ mcp/               # Model Context Protocol
|   └─ mcp.ts           # MCP 定义
|   └─ client.ts        # MCP 客户端
|   └─ server.ts        # MCP 服务器
|   └─ transport/       # 传输层
|
| └─ config/            # 配置管理
|   └─ config.ts        # 配置定义和加载
|   └─ ...
|
| └─ server/            # HTTP 服务器
|   └─ server.ts        # 服务器入口
|   └─ routes/          # API 路由
|
| └─ plugin/            # 插件系统

```

```

| | └─ plugin.ts          # 插件管理
| |   └─ builtin/         # 内置插件
| |
| └─ storage/             # 存储层
| |   └─ storage.ts       # 存储接口
| |     └─ ...
| |
| |
| └─ auth/                # 认证管理
| |   └─ auth.ts          # 认证接口
| |     └─ oauth.ts       # OAuth 实现
| |
| |
| └─ permission/          # 权限控制
| |   └─ next.ts          # 新版权限系统
| |     └─ index.ts       # 旧版权限系统
| |
| |
| └─ project/             # 项目管理
| |   └─ instance.ts      # 项目实例
| |     └─ worktree.ts    # 工作树管理
| |
| |
| └─ file/                # 文件操作
| |   └─ ripgrep.ts       # Ripgrep 集成
| |     └─ ...
| |
| |
| └─ util/                # 工具函数
| |   └─ log.ts           # 日志系统
| |   └─ bus.ts           # 事件总线
| |     └─ ...
| |
| |
| └─ installation.ts      # 安装信息
|
└─ package.json

```

## 4. 架构设计原则

### 4.1 模块化设计

每个模块职责单一，通过明确的接口进行通信。核心模块包括： - **CLI**: 命令行入口 - **Config**: 配置管理 - **Provider**: LLM 提供商抽象 - **Agent**: 智能体管理 - **Tool**: 工具系统 - **Session**: 会话管理 - **LLM**: 大模型通信 - **Processor**: 流式处理

### 4.2 事件驱动架构

使用自定义事件系统 (Bus) 实现模块间解耦通信：

```
// 发布事件
Bus.publish(MessageV2.Event.Updated, { info: msg })

// 订阅事件
Bus.subscribe(MessageV2.Event.Updated, ({ info }) => {

  // 处理事件

})
```

### 4.3 插件化扩展

支持在关键点通过插件扩展功能： - `tool.execute.before` - 工具执行前 - `tool.execute.after` - 工具执行后 - `experimental.chat.system.transform` - 系统提示词转换 - `experimental.text.complete` - 文本完成

### 4.4 类型安全

使用 Zod 进行运行时类型验证：

```
export const Info = z.object({
  id: z.string(),
  name: z.string(),
  permission: PermissionNext.Ruleset,
  model: z.object({
    modelID: z.string(),
    providerID: z.string(),
  }).optional(),
})
```

---

## 5. 核心模块详解

### 5.1 CLI 模块 ( `src/cli/` )

功能: 命令行入口和参数解析

主入口文件: `src/index.ts`

```
// CLI 命令列表
const cli = yargs(hideBin(process.argv))
  .scriptName("opencode")
  .command(RunCommand) // 运行 OpenCode
  .command(AuthCommand) // 认证管理
  .command(AgentCommand) // Agent 管理
  .command(GenerateCommand) // 生成配置
  .command(ServeCommand) // 启动服务
  .command(SessionCommand) // 会话管理
  .command(ModelsCommand) // 模型列表
  .command(McpCommand) // MCP 管理
  // ... 更多命令
```

核心流程: 1. 初始化日志系统 2. 解析命令行参数 3. 调用相应命令处理器 4. 错误处理和格式化输出

---



## 5.2 Config 模块 ( `src/config/` )

功能: 多层次配置管理

配置加载顺序 (从高到低):

1. 远程配置 - `.well-known/opencode`
2. 全局配置 - `~/.config/opencode/config.json`
3. 自定义配置 - `OPENCODE_CONFIG` 环境变量
4. 项目配置 - `opencode.json`
5. 目录配置 - `.opencode/` 目录
6. 内联配置 - `OPENCODE_CONFIG_CONTENT` 环境变量

配置结构:

```
export const Info = z.object({
  agent: z.record(Agent),    // 智能体配置
  provider: z.record(Provider), // 提供商配置
  model: z.string(),         // 默认模型
  permission: Permission,    // 权限控制
  plugin: z.string().array(), // 插件列表
  autoConfirm: z.boolean(),  // 自动确认
  fork: z.boolean(),         // Fork 模式
  suggestCommands: z.boolean(), // 命令建议
  storage: z.enum(["memory", "disk"]), // 存储类型
})
```

---

## 5.3 Provider 模块 ( `src/provider/` )

功能: LLM 提供商抽象和模型管理

核心接口:

```

export namespace Provider {
  export const Info = z.object({
    id: z.string(),
    base: z.string(),
    headers: z.record(z.string()).optional(),
    options: z.record(z.string()).optional(),
  })

  export const Model = z.object({
    providerID: z.string(),
    modelID: z.string(),
    api: z.object({
      id: z.string(),
      base: z.string().optional(),
    }),
    capabilities: z.object({
      temperature: z.boolean(),
      toolcall: z.boolean(),
      interleaved: z.boolean(),
    }),
    cost: z.object({
      input: z.number(),
      output: z.number(),
    }),
    limit: z.object({
      context: z.number(),
    }),
    options: z.record(z.string()).optional(),
  })
}

```

内置提供商:

| 提供商       | 包名                | 模型示例              |
|-----------|-------------------|-------------------|
| OpenAI    | @ai-sdk/openai    | gpt-4, gpt-5      |
| Anthropic | @ai-sdk/anthropic | claude-3-5-sonnet |
| Google    | @ai-sdk/google    | gemini-2.0-flash  |
| Groq      | @ai-sdk/groq      | llama-3.3-70b     |
| Ollama    | @ai-sdk/ollama    | 本地模型              |
| OpenCode  | @opencode-ai/sdk  | opencode 模型       |

---

## 5.4 Agent 模块 ( `src/agent/` )

功能: AI 智能体角色定义和管理

核心数据结构:

```

export namespace Agent {
  export const Info = z.object({
    name: z.string(),
    description: z.string().optional(),
    mode: z.enum(["subagent", "primary", "all"]),
    native: z.boolean().optional(),
    hidden: z.boolean().optional(),
    temperature: z.number().optional(),
    topP: z.number().optional(),
    color: z.string().optional(),
    permission: PermissionNext.Ruleset,
    model: z.object({
      modelID: z.string(),
      providerID: z.string(),
    }).optional(),
    variant: z.string().optional(),
    prompt: z.string().optional(),
    options: z.record(z.string(), z.any()),
    steps: z.number().int().positive().optional(),
  })
}

```

## 内置 Agent:

| Agent             | 模式       | 用途       | 特点               |
|-------------------|----------|----------|------------------|
| <b>build</b>      | primary  | 默认 Agent | 执行基于权限的工具        |
| <b>plan</b>       | primary  | 计划模式     | 禁止编辑工具，只允许计划     |
| <b>general</b>    | subagent | 通用 Agent | 研究和多步骤任务，禁用 Todo |
| <b>explore</b>    | subagent | 探索 Agent | 快速探索代码库，只允许只读操作  |
| <b>title</b>      | primary  | 标题生成     | 隐藏 Agent，禁用所有工具  |
| <b>summary</b>    | primary  | 总结 Agent | 隐藏 Agent，生成会话总结  |
| <b>compaction</b> | primary  | 压缩 Agent | 隐藏 Agent，压缩对话历史  |

## 默认权限配置:

```
const defaults = PermissionNext.fromConfig({
  "**": "allow",      // 默认允许所有操作
  doom_loop: "ask",   // 循环检测需要询问
  external_directory: {
    "**": "ask",       // 外部目录需要询问
  },
  question: "deny",   // 默认禁止提问
  plan_enter: "deny", // 默认禁止进入计划模式
  plan_exit: "deny",  // 默认禁止退出计划模式
  read: {
    "**": "allow",     // 默认允许读取
    "**.env": "ask",   // 环境变量文件需要询问
  },
})
```

---

## 5.5 Tool 模块 ( `src/tool/` )

功能: 工具定义、注册和执行

工具接口定义 ( `tool.ts` ):

```

export namespace Tool {
  export interface InitContext {
    agent?: Agent.Info
  }

  export type Context<M extends Metadata = Metadata> = {
    sessionID: string
    messageID: string
    agent: string
    abort: AbortSignal
    callID?: string
    extra?: { [key: string]: any }
    messages: MessageV2.WithParts[]
    metadata(input: { title?: string; metadata?: M }): void
    ask(input: Omit<PermissionNext.Request, "id" | "sessionID" | "tool">): Promise<void>
  }

  export interface Info<Parameters extends z.ZodType = z.ZodType, M extends Metadata = Metadata> {
    id: string
    init: (ctx?: InitContext) => Promise<{
      description: string
      parameters: Parameters
      execute(args: z.infer<Parameters>, ctx: Context): Promise<{
        title: string
        metadata: M
        output: string
        attachments?: MessageV2.FilePart[]
      }>
      formatValidationError?(error: z.ZodError): string
    }>
  }

  export function define<Parameters extends z.ZodType, Result extends Metadata>(
    id: string,
    init: Info<Parameters, Result>["init"] | Awaited<ReturnType<Info<Parameters, Result>["init"]>>,
  ): Info<Parameters, Result>
}

```

内置工具列表:

| 工具                       | 功能   | 参数                                          |
|--------------------------|------|---------------------------------------------|
| <code>read</code>        | 读取文件 | filePath, offset?, limit?                   |
| <code>write</code>       | 写入文件 | content, filePath                           |
| <code>edit</code>        | 编辑文件 | filePath, oldString, newString, replaceAll? |
| <code>bash</code>        | 执行命令 | command, description, timeout?, workdir?    |
| <code>glob</code>        | 文件匹配 | pattern, path?                              |
| <code>grep</code>        | 内容搜索 | pattern, path?, include?                    |
| <code>task</code>        | 任务管理 | (多个子命令)                                     |
| <code>websearch</code>   | 网络搜索 | query                                       |
| <code>webfetch</code>    | 网页抓取 | url                                         |
| <code>skill</code>       | 技能执行 | skill, args?                                |
| <code>batch</code>       | 批量操作 | tasks                                       |
| <code>apply_patch</code> | 应用补丁 | diff                                        |

工具特性: 1. 参数验证: 使用 `Zod` 进行运行时类型检查 2. 输出截断: 自动处理大输出, 保存到文件 3. 权限控制: 通过 `ctx.ask()` 请求权限 4. 错误处理: 统一的错误格式化和处理 5. 插件支持: 工具执行前后触发插件事件

## 5.6 Session 模块 ( `src/session/` )

功能: 会话生命周期管理

核心数据结构:

```

export namespace Session {
  export const Info = z.object({
    id: Identifier.schema("session"),
    slug: z.string(),
    title: z.string(),
    permission: PermissionNext.Ruleset,
    time: z.object({
      created: z.number(),
      updated: z.number(),
    }),
  })

  // 消息结构
  export type Part =
    | TextPart // 文本片段
    | ReasoningPart // 推理片段
    | ToolPart // 工具片段
    | FilePart // 文件片段
    | StepStartPart // 步骤开始
    | StepFinishPart // 步骤结束

  export interface TextPart {
    id: string
    messageID: string
    sessionID: string
    type: "text"
    text: string
    time?: { start: number; end?: number }
    metadata?: Record<string, any>
  }

  export interface ToolPart {
    id: string
    messageID: string
    sessionID: string
    type: "tool"
    callID: string
  }

```



```
tool:string

state:ToolState

metadata?:Record<string,any>
}

export type ToolState=
|{status:"pending",input:{}}
|{status:"running",input:{...},time:{start}}
|{status:"completed",input:{...},output:string,time:{start,end}}
|{status:"error",error:string}
}
```

---

## 5.7 LLM 模块 ( `src/session/llm.ts` )

功能: 大模型通信

核心函数:

```

export async function stream(input: StreamInput){
  // 1. 并行获取配置
  const [language, cfg, provider, auth] = await Promise.all([
    Provider.getLanguage(input.model),
    Config.get(),
    Provider.getProvider(input.model.providerID),
    Auth.get(input.model.providerID),
  ])

  // 2. 构建系统提示词
  const system = [
    ...(input.agent.prompt ? [input.agent.prompt] : []),
    ...input.system,
    ...(input.user.system ? [input.user.system] : []),
  ].filter((x) => x).join("\n")

  // 3. 解析工具
  const tools = await resolveTools(input)

  // 4. 调用 Vercel AI SDK
  return streamText({
    model: wrapLanguageModel({ model: language }),
    messages: [
      ...system.map((x): ModelMessage => ({ role: "system", content: x })),
      ...input.messages,
    ],
    tools,
    maxOutputTokens,
    abortSignal: input.abort,
    temperature: input.agent.temperature,
    experimental_repairToolCall(failed){
      // 工具调用名称大小写修复
      const lower = failed.toolCall.toolName.toLowerCase()
      if (lower !== failed.toolCall.toolName && tools[lower]){
        return { ...failed.toolCall, toolName: lower }
      }
    }
  })
  return { ...failed.toolCall, toolName: "invalid" }
}

```

```
},
}))
}
```

## 5.8 Processor 模块 ( `src/session/processor.ts` )

功能: 流式事件处理引擎

处理的事件类型:

| 事件类型                         | 处理动作      |
|------------------------------|-----------|
| <code>text-start</code>      | 创建文本 Part |
| <code>text-delta</code>      | 增量更新文本内容  |
| <code>text-end</code>        | 完成文本 Part |
| <code>reasoning-start</code> | 创建推理 Part |
| <code>reasoning-delta</code> | 更新推理内容    |
| <code>reasoning-end</code>   | 完成推理 Part |
| <code>tool-call</code>       | 执行工具调用    |
| <code>tool-result</code>     | 更新工具完成状态  |
| <code>tool-error</code>      | 处理工具错误    |
| <code>finish-step</code>     | 完成处理步骤    |
| <code>error</code>           | 错误处理和重试   |

循环检测机制:

```

const DOOM_LOOP_THRESHOLD = 3

// 在工具调用时检测
case "tool-call":{
  const lastThree = parts.slice(-DOOM_LOOP_THRESHOLD)

  if (
    lastThree.length === DOOM_LOOP_THRESHOLD &&
    lastThree.every(
      (p) =>
        p.type === "tool" &&
        p.tool === value.toolName &&
        p.state.status !== "pending" &&
        JSON.stringify(p.state.input) === JSON.stringify(value.input),
    )
  ){
    // 检测到循环，请求权限确认
    await PermissionNext.ask({
      permission: "doom_loop",
      patterns: [value.toolName],
      sessionId: input.assistantMessage.sessionID,
      metadata: { tool: value.toolName, input: value.input },
      always: [value.toolName],
      ruleset: agent.permission,
    })
  }
  break
}

```

## 5.9 System Prompt 模块 ( `src/session/system.ts` )

功能: 系统提示词构建

构建顺序:

```

export namespace SystemPrompt {
  // 1. 基础指令 - codex_header.txt
  export function instructions() {
    return PROMPT_CODEX.trim()
  }

  // 2. 提供商特定提示
  export function provider(model: Provider.Model) {
    if (model.api.id.includes("gpt-5")) return [PROMPT_CODEX]
    if (model.api.id.includes("gpt-") || model.api.id.includes("o1") || model.api.id.includes("o3"))
      return [PROMPT_BEAST]
    if (model.api.id.includes("gemini-")) return [PROMPT_GEMINI]
    if (model.api.id.includes("claude")) return [PROMPT_ANTHROPIC]
    if (model.api.id.toLowerCase().includes("trinity")) return [PROMPT_TRINITY]
    return [PROMPT_ANTHROPIC_WITHOUT_TODO]
  }

  // 3. 环境信息
  export async function environment(model: Provider.Model) {
    const project = Instance.project
    return [
      [
        `You are powered by the model named ${model.api.id}.`,
        `Here is some useful information about the environment you are running in:`,
        ``,
        `  Working directory: ${Instance.directory}`,
        `  Is directory a git repo: ${project.vcs === "git" ? "yes" : "no"}`,
        `  Platform: ${process.platform}`,
        `  Today's date: ${new Date().toLocaleDateString()}`,
        `</env>`,
      ].join("\n"),
    ]
  }
}

```

核心指令文件 ( `codex_header.txt` ):

You are OpenCode, the best coding agent on the planet.

You are an interactive CLI tool that helps users with software engineering tasks.

#### ## Editing constraints

- Default to ASCII when editing or creating files.
- Only add comments if they are necessary.
- Try to use `apply_patch` for single file edits.

#### ## Tool usage

- Prefer specialized tools over shell for file operations.
- Use Bash for terminal operations.
- Run tool calls in parallel when possible.

#### ## Git and workspace hygiene

- You may be in a dirty git worktree.
- NEVER revert existing changes you did not make unless explicitly requested.
- NEVER use destructive commands like `git reset --hard` unless specifically requested.

#### ## Frontend tasks

- Avoid collapsing into bland, generic layouts.
- Use expressive fonts and avoid default stacks.
- Choose a clear visual direction; avoid purple-on-white defaults.
- Use meaningful animations instead of generic micro-motions.

#### ## Presenting your work

- Be very concise; friendly coding teammate tone.
- Do the work without asking questions.
- Only ask when truly blocked.
- For code changes: lead with explanation, then details.
- File References: use inline code for paths (e.g., `src/app.ts:42`)

---

## 5.10 Permission 模块 ( `src/permission/` )

功能: 细粒度权限控制系统

## 核心数据结构:

```
export namespace PermissionNext {  
  
  // 权限动作类型  
  
  export const Action = z.enum(["allow", "deny", "ask"])  
  
  export type Action = z.infer<typeof Action>  
  
  
  // 权限规则  
  
  export const Rule = z.object({  
  
    permission: z.string(),  
  
    pattern: z.string(),  
  
    action: Action,  
  
  })  
  
  export type Rule = z.infer<typeof Rule>  
  
  
  // 权限规则集  
  
  export const Ruleset = Rule.array()  
  
  export type Ruleset = z.infer<typeof Ruleset>  
  
  
  // 权限请求  
  
  export const Request = z.object({  
  
    id: Identifier.schema("permission"),  
  
    sessionID: Identifier.schema("session"),  
  
    permission: z.string(),  
  
    patterns: z.string().array(),  
  
    metadata: z.record(z.string(), z.any()),  
  
    always: z.string().array(),  
  
    tool: z.object({  
  
      messageID: z.string(),  
  
      callID: z.string(),  
  
    }).optional(),  
  
  })  
  
}
```

## 权限类型:

| 权限                 | 说明     |
|--------------------|--------|
| read               | 文件读取   |
| edit               | 文件编辑   |
| bash               | 命令执行   |
| glob               | 文件模式匹配 |
| grep               | 内容搜索   |
| external_directory | 外部目录访问 |
| webfetch           | 网络抓取   |
| websearch          | 网络搜索   |
| doom_loop          | 循环检测   |
| question           | 提问     |
| plan_enter         | 进入计划模式 |
| plan_exit          | 退出计划模式 |

权限评估:

```
export function evaluate(permission: string, pattern: string, ...rulesets: Ruleset[]): Rule {
  const merged = merge(...rulesets)
  const match = merged.findLast(
    (rule) => Wildcard.match(permission, rule.permission) && Wildcard.match(pattern, rule.pattern),
  )
  return match ?? { action: "ask", permission, pattern: "*" }
}
```

错误类型:



```

// 用户拒绝 - 中止执行
export class RejectedError extends Error {
  constructor() {
    super(`The user rejected permission to use this specific tool call.`)
  }
}

// 用户拒绝并提供反馈 - 继续执行
export class CorrectedError extends Error {
  constructor(message: string) {
    super(`The user rejected permission with feedback: ${message}`)
  }
}

// 配置规则拒绝 - 中止执行
export class DeniedError extends Error {
  constructor(public readonly ruleset: Ruleset) {
    super(`Rule prevents tool usage. Relevant rules: ${JSON.stringify(ruleset)}`)
  }
}

```

## 5.11 MCP 模块 ( `src/mcp/` )

功能: Model Context Protocol 实现

**MCP** 类型:

```
export const Mcp = z.discriminatedUnion("type", [
  z.object({
    type: z.literal("local"),
    command: z.string(),
    args: z.string().array(),
    env: z.record(z.string()).optional(),
  }),
  z.object({
    type: z.literal("remote"),
    url: z.string(),
    headers: z.record(z.string()).optional(),
  }),
])
```

支持的传输方式: - **Local**: 本地命令启动的 MCP 服务器 - **Remote**: HTTP 远程 MCP 服务器 - **OAuth**: 支持 OAuth 认证的 MCP 服务器

## 5.12 Plugin 模块 ( `src/plugin/` )

功能: 插件系统

插件钩子:

| 钩子                                              | 触发时机    |
|-------------------------------------------------|---------|
| <code>tool.execute.before</code>                | 工具执行前   |
| <code>tool.execute.after</code>                 | 工具执行后   |
| <code>experimental.chat.system.transform</code> | 系统提示词转换 |
| <code>experimental.text.complete</code>         | 文本完成    |

插件 **API**:

```
export namespace Plugin {  
  export function transform(hook: string, input: any): Promise<void>  
  export function trigger(hook: string, context: any, data: any): Promise<any>  
  export function register(plugin: Plugin.Info): void  
}
```

---

## 5.13 Storage 模块 ( `src/storage/` )

功能: 数据持久化

存储接口:

```
export namespace Storage {  
  export function write(path: string[], value: any): Promise<void>  
  export function read<T>(path: string[]): Promise<T | undefined>  
  export function exists(path: string[]): Promise<boolean>  
  export function remove(path: string[]): Promise<void>  
  export function list(path: string[]): Promise<string[]>  
}
```

存储结构:

```
~/.opencode/  
├─ sessions/  
|   └─ {projectId}/  
|       └─ {sessionId}.json  
├─ messages/  
|   └─ {projectId}/  
|       └─ {sessionId}/  
|           └─ {messageID}.json  
└─ config.json
```

## 5.14 Auth 模块 ( `src/auth/` )

功能: 认证管理

认证类型:

```
export namespace Auth {  
  export const Info = z.discriminatedUnion("type", [  
    z.object({  
      type: z.literal("oauth"),  
      accessToken: z.string(),  
      refreshToken: z.string().optional(),  
      expiresAt: z.number().optional(),  
    }),  
    z.object({  
      type: z.literal("apiKey"),  
      apiKey: z.string(),  
    })  
  ])  
}
```

支持的平台: - OpenAI OAuth - GitHub OAuth - 自定义 API Key

---

## 5.15 Server 模块 ( `src/server/` )

功能: HTTP 服务器和 API

**API** 路由:

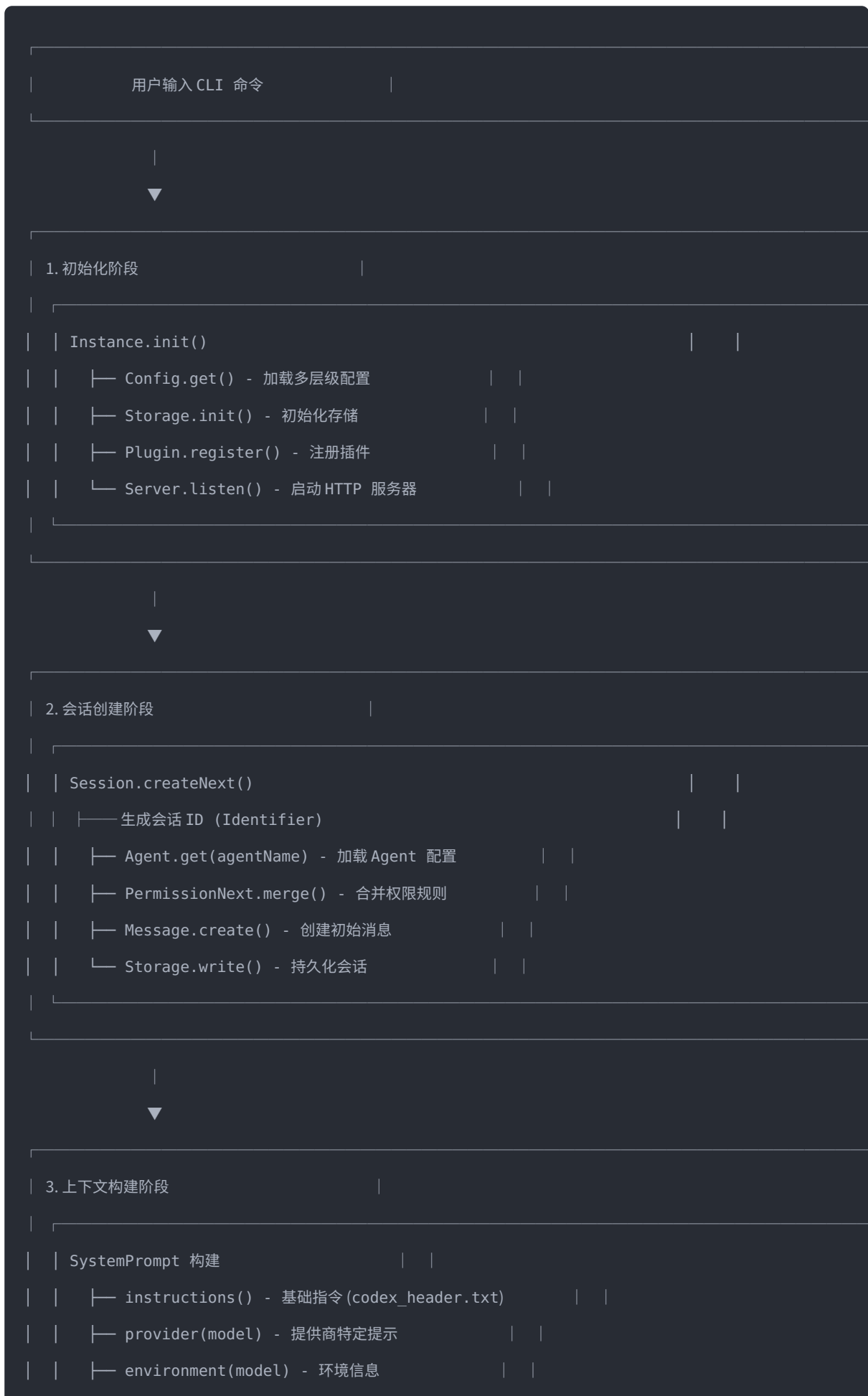
| 路由                    | 功能     |
|-----------------------|--------|
| <code>/global</code>  | 全局配置   |
| <code>/auth</code>    | 认证管理   |
| <code>/session</code> | 会话管理   |
| <code>/project</code> | 项目管理   |
| <code>/file</code>    | 文件操作   |
| <code>/config</code>  | 配置管理   |
| <code>/pty</code>     | 终端会话   |
| <code>/mcp</code>     | MCP 服务 |

**WebSocket** 通信: - 会话状态更新 - 工具执行结果 - 通知消息

---

## 6. 请求处理完整流程

### 6.1 流程图



```
| | |— agent.prompt - Agent 自定义提示 | |
| | |— user.system - 用户自定义提示 | |
```



#### 4. 工具解析阶段

```
| | ToolRegistry.tools() | |
| | |— 获取所有工具列表 | |
| | |— 根据 Agent 权限过滤工具 | |
| | |— 根据模型能力过滤工具 | |
| | |— 并行初始化工具 | |
| | |— 转换为 AI SDK 格式 | |
```



#### 5. LLM 通信阶段

```
| | LLM.stream() | |
| | |— Provider.getLanguage() - 获取语言模型 | |
| | |— 构建系统提示词 | |
| | |— Plugin.transform() - 应用插件转换 | |
| | |— resolveTools() - 解析工具 | |
| | |— streamText() - 调用 Vercel AI SDK | |
| | |— 返回流式响应(fullStream) | |
```



#### 6. 会话处理阶段

```
| | for await (const value of stream.fullStream) | |
```



```

| | |
| | |   ↳ text-start   → Session.updatePart() 创建文本 Part
| | |   |
| | |   |   Bus.publish(PartCreated)
| | |
| | |   ↳ text-delta   → Session.updatePart() 增量更新文本
| | |   |
| | |   |   Bus.publish(PartUpdated)
| | |
| | |   ↳ text-end     → Session.updatePart() 完成文本 Part
| | |   |
| | |   |   Plugin.trigger(text.complete)
| | |
| | |
| | |   ↳ tool-call     → 检测循环 (doom_loop)
| | |   |
| | |   |   PermissionNext.ask() 权限检查
| | |   |
| | |   |   Tool.execute() 执行工具
| | |   |
| | |   |   Plugin.trigger(tool.execute.before)
| | |   |
| | |   |   Plugin.trigger(tool.execute.after)
| | |
| | |
| | |   ↳ tool-result   → Session.updatePart() 更新工具状态
| | |   |
| | |   |   Bus.publish(PartUpdated)
| | |
| | |
| | |   ↳ tool-error    → Session.updatePart() 更新错误状态
| | |   |
| | |   |   检查是否需要中断 (blocked)
| | |
| | |
| | |   ↳ reasoning-*   → 处理推理过程 (Claude 模型)
| | |
| | |
| | |   ↳ finish-step    → 完成处理步骤
| | |
| | |   |   Session.updateMessage() 更新消息
| | |
| | |   |   Bus.publish(MessageUpdated)

```



## 7. 结果返回阶段

```

| | |   ↳ Storage.write() - 持久化消息历史
| | |   ↳ SessionStatus.set() - 更新会话状态
| | |   ↳ 返回给用户

```

等待下一个用户输入

## 6.2 时序图

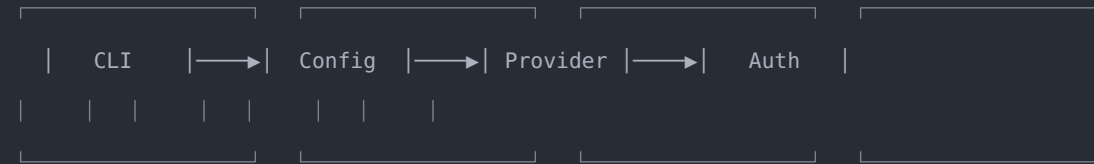




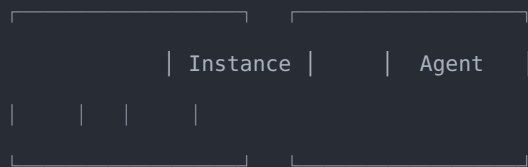
## 7. 数据流与模块关系

### 7.1 模块依赖关系图

模块依赖关系图

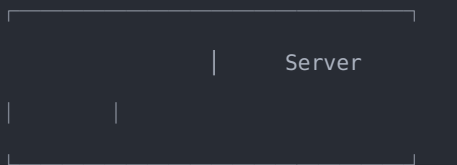


Module dependencies: CLI, Config, Provider, Auth



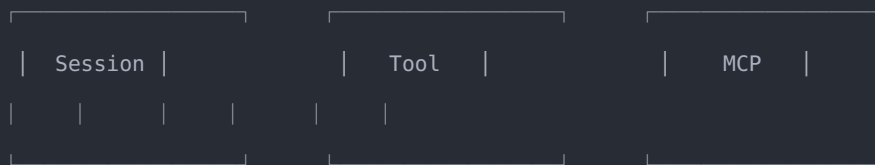
Module dependencies: Instance, Agent

Module dependencies: Instance, Agent



Module dependencies: Server

Module dependencies: Session, Tool, MCP



Module dependencies: Session, Tool, MCP



Module dependencies: Session, Tool, MCP





## 8. 设计模式总结

### 8.1 架构模式

| 模式     | 应用场景      | 位置                           |
|--------|-----------|------------------------------|
| 事件驱动架构 | 模块间通信     | <code>src/util/bus.ts</code> |
| 插件化架构  | 功能扩展      | <code>src/plugin/</code>     |
| 分层架构   | 职责分离      | 整体项目结构                       |
| 微内核架构  | 核心系统 + 插件 | CLI + 工具/插件                  |

### 8.2 设计模式

| 模式     | 应用场景        | 示例                                                          |
|--------|-------------|-------------------------------------------------------------|
| 工厂模式   | 创建工具、提供商    | <code>Tool.define()</code> , <code>Provider.create()</code> |
| 策略模式   | 不同的 LLM 提供商 | <code>Provider</code> 接口                                    |
| 观察者模式  | 事件系统        | <code>Bus.publish()</code> , <code>Bus.subscribe()</code>   |
| 中间件模式  | 请求/响应处理     | <code>wrapLanguageModel()</code>                            |
| 单例模式   | 全局配置        | <code>Config.get()</code> , <code>Instance.state()</code>   |
| 构建器模式  | 复杂对象构建      | <code>Agent.Info</code> 构建                                  |
| 命令模式   | CLI 命令      | <code>defineCliCommand()</code>                             |
| 适配器模式  | MCP 集成      | <code>MCP 适配器</code>                                        |
| 状态模式   | 会话状态        | <code>SessionStatus.set()</code>                            |
| 模板方法模式 | 工具执行流程      | <code>Tool.execute()</code>                                 |

### 8.3 核心设计原则

#### 1. 单一职责原则 (SRP)

- 每个模块只负责一个功能领域
- 例如: `Tool` 只负责工具定义 , `Storage` 只负责数据持久化

## 2. 开放封闭原则 (OCP)

- 通过插件系统扩展功能
- 新增工具不需要修改核心代码

## 3. 依赖倒置原则 (DIP)

- 模块依赖抽象接口而非具体实现
- 例如: `Provider` 接口定义了所有提供商必须实现的方法

## 4. 接口隔离原则 (ISP)

- 工具接口精简, 只包含必要方法
- `Tool.Info` 接口只定义 `id` 和 `init`

## 5. 里氏替换原则 (LSP)

- 所有 `Provider` 实现可以互相替换
- 所有 `Tool` 实现遵循相同的接口约定

# 附录

## A. 关键常量

| 常量                               | 值      | 说明              |
|----------------------------------|--------|-----------------|
| <code>DOOM_LOOP_THRESHOLD</code> | 3      | 循环检测阈值          |
| <code>MAX_OUTPUT_LENGTH</code>   | 50000  | 最大输出长度          |
| <code>DEFAULT_TIMEOUT</code>     | 120000 | 默认超时时间 (2分钟)    |
| <code>MAX_FILE_SIZE</code>       | 50000  | 最大文件读取大小 (50KB) |

## B. 环境变量

| 变量                                   | 说明             |
|--------------------------------------|----------------|
| <code>OPENCODE</code>                | 标识 OpenCode 环境 |
| <code>AGENT</code>                   | 标识 Agent 环境    |
| <code>OPENCODE_CONFIG</code>         | 自定义配置路径        |
| <code>OPENCODE_CONFIG_CONTENT</code> | 内联配置内容         |
| <code>OPENCODE_FORK</code>           | Fork 模式        |

## C. 文件路径约定

| 路径                               | 说明     |
|----------------------------------|--------|
| <code>~/.config/opencode/</code> | 全局配置目录 |
| <code>~/.opencode/</code>        | 数据存储目录 |
| <code>.opencode/</code>          | 项目配置目录 |
| <code>opencode.json</code>       | 项目配置文件 |

## D. 相关资源

- **Vercel AI SDK**: <https://sdk.vercel.ai/docs>
- **Model Context Protocol**: <https://modelcontextprotocol.io>
- **Ripgrep**: <https://github.com/BurntSushi/ripgrep>
- **Hono**: <https://hono.dev>

---

### 文档结束

本文档由 OpenCode 项目源代码分析生成，详细描述了项目的完整架构、模块实现和请求处理流程。